

Clean Architecture, Singleton & Sealed Class, [ASP.NET](#) Core Lifecycle, .NET Core vs .NET Framework, Microservices Communication & Middleware

*Clean Architecture, Singleton vs Static Class, [ASP.NET](#) Core Lifecycle, .NET Core vs .NET Framework, Microservices Communication, and Middleware.
Read it once. Understand it forever.*

Table of Contents

1. [Clean Architecture](#)
 2. [Singleton Pattern vs Static Class](#)
 3. [Lifecycle of ASP.NET Core Application](#)
 4. [.NET Core vs .NET Framework](#)
 5. [Microservices and Communication](#)
 6. [Middleware in ASP.NET Core](#)
 7. [Interview Questions and Answers](#)
-

1. Clean Architecture

The Problem It Solves

Picture a codebase that has been worked on by ten people over three years. The database logic is sprinkled inside controllers. Business rules reference specific UI models. A unit test requires a live database connection to run. Changing the database means touching fifty files. This is what happens when architecture is not thought about.

Clean Architecture is a way of organising your code so that:

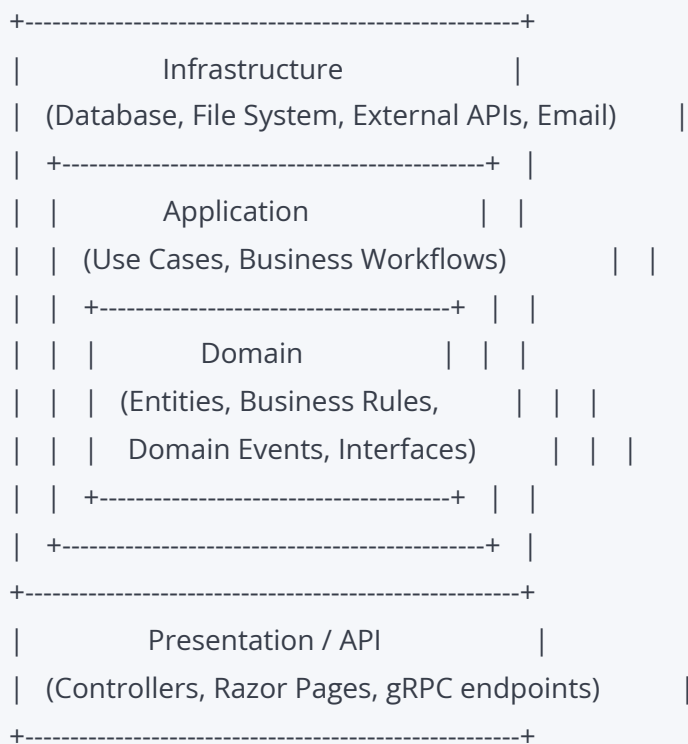
- Business rules are the center of the universe
- The database, UI, and external services are details that can be swapped out

- You can test your business logic without spinning up a database or a web server

The original idea comes from Robert C. Martin (Uncle Bob), and it builds on earlier ideas like Hexagonal Architecture and Onion Architecture. All of them share the same core belief: **the inner layers should never know about the outer layers.**

The Layers

Think of it as an onion. The innermost layer is the most important and the most stable. The outer layers are implementation details.



The Dependency Rule: Source code dependencies must point inward. The Domain knows nothing about Application. Application knows nothing about Infrastructure. Infrastructure knows nothing about Presentation. The arrow goes one way: inward.

Layer by Layer

Domain Layer — The Heart

This is the purest layer. It contains:

- **Entities:** Core business objects with identity and behaviour
- **Value Objects:** Objects defined by their attributes, not identity (Money, Address)

- **Domain Events:** Things that happened in the domain (OrderPlaced, PaymentReceived)
- **Interfaces:** Contracts that outer layers must fulfil
- **Business Rules:** Logic that is always true regardless of the application

```
// Domain/Entities/Order.cs
// No references to EF Core, no references to HttpClient, no references to anything external
public class Order
{
    public Guid Id { get; private set; }
    public string CustomerId { get; private set; }
    public List<OrderItem> Items { get; private set; } = new();
    public OrderStatus Status { get; private set; }
    public decimal TotalAmount => Items.Sum(i => i.Quantity * i.UnitPrice);

    private Order() { } // For EF Core

    public static Order Create(string customerId)
    {
        if (string.IsNullOrEmpty(customerId))
            throw new DomainException("Customer ID cannot be empty.");

        return new Order
        {
            Id = Guid.NewGuid(),
            CustomerId = customerId,
            Status = OrderStatus.Pending
        };
    }

    public void AddItem(string productId, int quantity, decimal unitPrice)
    {
        if (Status != OrderStatus.Pending)
            throw new DomainException("Cannot add items to a non-pending order.");

        Items.Add(new OrderItem(productId, quantity, unitPrice));
    }

    public void Confirm()
    {

```

```

        if (!Items.Any())
            throw new DomainException("Cannot confirm an empty order.");

        Status = OrderStatus.Confirmed;
    }
}

```

Notice: no `using Microsoft.EntityFrameworkCore` . No `using System.Net.Http` . Nothing from the outside world. This class can be tested by just writing `new Order(...)` .

Application Layer — The Orchestrator

This layer contains your use cases. Each use case is a specific thing the application can do: Place an Order, Cancel a Subscription, Get User Profile. It orchestrates domain objects and calls interfaces defined in the Domain layer.

It knows about the Domain but does not know about the database, HTTP, or any framework.

```

// Application/Orders/Commands/PlaceOrderCommand.cs
public record PlaceOrderCommand(string CustomerId, List<OrderItemDto> Items);

// Application/Orders/Commands/PlaceOrderCommandHandler.cs
public class PlaceOrderCommandHandler
{
    private readonly IOrderRepository _orderRepository;
    private readonly IEventPublisher _eventPublisher;

    public PlaceOrderCommandHandler(
        IOrderRepository orderRepository,
        IEventPublisher eventPublisher)
    {
        _orderRepository = orderRepository;
        _eventPublisher = eventPublisher;
    }

    public async Task<Guid> Handle(PlaceOrderCommand command, CancellationToken
    {
        var order = Order.Create(command.CustomerId);
    }
}

```

```

        foreach (var item in command.Items)
            order.AddItem(item.ProductId, item.Quantity, item.UnitPrice);

        order.Confirm();

        await _orderRepository.AddAsync(order, ct);

        await _eventPublisher.PublishAsync(new OrderPlacedEvent(order.Id), ct);

        return order.Id;
    }
}

```

`IOrderRepository` and `IEventPublisher` are interfaces defined in the Domain or Application layer. The Application layer does not know whether `IOrderRepository` is backed by SQL Server or MongoDB. It does not care.

Infrastructure Layer — The Details

This is where the real implementations live. This layer knows about EF Core, SQL Server, Redis, SendGrid, AWS S3, and everything else external.

```

// Infrastructure/Persistence/OrderRepository.cs
public class SqlOrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;

    public SqlOrderRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task AddAsync(Order order, CancellationToken ct)
    {
        await _context.Orders.AddAsync(order, ct);
        await _context.SaveChangesAsync(ct);
    }

    public async Task<Order?> GetByIdAsync(Guid id, CancellationToken ct)
    {

```

```

        return await _context.Orders
            .Include(o => o.Items)
            .FirstOrDefaultAsync(o => o.Id == id, ct);
    }
}

```

```

// Infrastructure/DependencyInjection.cs
public static class InfrastructureServiceCollectionExtensions
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration config)
    {
        services.AddDbContext<AppDbContext>(options =>
            options.UseSqlServer(config.GetConnectionString("Default")));

        services.AddScoped<IOrderRepository, SqlOrderRepository>();
        services.AddScoped<IEventPublisher, RabbitMqEventPublisher>();

        return services;
    }
}

```

Presentation Layer — The Entry Point

Controllers, Razor Pages, gRPC endpoints, or Minimal API handlers. They receive HTTP requests, translate them to commands or queries, and return HTTP responses. They know about the Application layer but not about Domain internals or Infrastructure.

```

// Presentation/Controllers/OrdersController.cs
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly PlaceOrderCommandHandler _handler;

    public OrdersController(PlaceOrderCommandHandler handler)
    {
        _handler = handler;
    }
}

```

```
}

[HttpPost]
public async Task<ActionResult> PlaceOrder([FromBody] PlaceOrderRequest request)
{
    var command = new PlaceOrderCommand(request.CustomerId, request.Items);
    var orderId = await _handler.Handle(command, HttpContext.RequestAborted);
    return CreatedAtAction(nameof(GetOrder), new { id = orderId }, null);
}
}
```

Project Structure in Practice

```
src/
  YourApp.Domain/
    Entities/
    ValueObjects/
    Events/
    Interfaces/
    Exceptions/

  YourApp.Application/
    Orders/
      Commands/
      Queries/
      DTOs/
    Common/
      Behaviours/ (cross-cutting: logging, validation)

  YourApp.Infrastructure/
    Persistence/
    Messaging/
    ExternalServices/
    DependencyInjection.cs

  YourApp.API/
    Controllers/
    Middleware/
    Program.cs
```

```
tests/  
YourApp.Domain.Tests/  
YourApp.Application.Tests/ (no DB required)  
YourApp.Integration.Tests/
```

CQRS with Clean Architecture

Clean Architecture pairs naturally with **CQRS (Command Query Responsibility Segregation)**, often implemented with MediatR.

CQRS says: separate the operations that change data (Commands) from the operations that read data (Queries). They have different requirements. Commands need validation and domain logic. Queries need raw performance.

```
// A Command changes state  
public record CreateProductCommand(string Name, decimal Price) : IRequest<Guid>;  
  
// A Query reads state  
public record GetProductByIdQuery(Guid Id) : IRequest<ProductDto>;
```

MediatR acts as the in-process message bus. The controller sends a command, MediatR finds the handler, calls it, and returns the result. The controller never knows which class actually handled it.

Why Clean Architecture Wins

Without Clean Architecture	With Clean Architecture
Changing the database is terrifying	Swap infrastructure, business logic untouched
Unit tests need a running database	Domain and Application tests need zero infrastructure
Business logic scattered everywhere	Business logic lives in one place
Tight coupling makes refactoring slow	Loose coupling through interfaces

2. Singleton Pattern vs Static Class

These two are often confused because they both give you “one thing shared everywhere.” But they are fundamentally different in what they are and what you can do with them.

Static Class — The Utility Belt

A static class is a compile-time construct. It cannot be instantiated. All its members are static. It is essentially a named bag of functions and constants. There is no object. There is no instance. There is nothing to inject, nothing to mock, nothing to inherit.

```
public static class MathUtils
{
    public static int Clamp(int value, int min, int max)
        => Math.Max(min, Math.Min(max, value));

    public static bool IsPrime(int n)
    {
        if (n < 2) return false;
        for (int i = 2; i <= Math.Sqrt(n); i++)
            if (n % i == 0) return false;
        return true;
    }
}

// Usage
int result = MathUtils.Clamp(150, 0, 100); // 100
bool prime = MathUtils.IsPrime(17); // true
```

Static classes are perfect for pure functions: input goes in, output comes out, no state is touched.

What a static class cannot do:

- It cannot implement an interface
- It cannot be passed as a parameter

- It cannot be swapped in a test
 - It cannot be inherited
 - It cannot participate in polymorphism
 - It cannot be registered in the DI container
-

Singleton Pattern — The One True Instance

The Singleton Pattern is a design pattern where a class is written such that only one instance can exist. Unlike a static class, it IS an object — it just happens to be created only once.

Classic manual singleton (not recommended in modern .NET):

```
public class ConfigurationManager
{
    private static ConfigurationManager? _instance;
    private static readonly object _lock = new object();

    private Dictionary<string, string> _settings = new();

    private ConfigurationManager()
    {
        // Load settings from file
        _settings["AppName"] = "MyApp";
        _settings["Version"] = "1.0.0";
    }

    public static ConfigurationManager Instance
    {
        get
        {
            if (_instance == null)
            {
                lock (_lock) // Thread safety
                {
                    if (_instance == null) // Double-check after acquiring lock
                        _instance = new ConfigurationManager();
                }
            }
            return _instance;
        }
    }
}
```

```

    }
}

public string Get(string key) => _settings.TryGetValue(key, out var val) ? val : string.Empty;
}

```

```
var name = ConfigurationManager.Instance.Get("AppName");
```

This is called **double-checked locking** and it is thread-safe.

Modern .NET approach — let the DI container manage it:

```

// Registration in Program.cs
builder.Services.AddSingleton<IConfigurationService, ConfigurationService>();

// The class itself is just a normal class
public class ConfigurationService : IConfigurationService
{
    private readonly Dictionary<string, string> _settings;

    public ConfigurationService(IConfiguration config)
    {
        _settings = config.GetSection("AppSettings").GetChildren()
            .ToDictionary(x => x.Key, x => x.Value ?? "");
    }

    public string Get(string key) => _settings.TryGetValue(key, out var val) ? val : "";
}

```

The container ensures only one instance is created. You benefit from singleton behaviour while keeping the class testable, interface-based, and injectable.

Lazy Initialization in Singletons

When you want the instance to be created only when first accessed, not at app startup:

```

public class HeavyResourceManager
{
    private static readonly Lazy<HeavyResourceManager> _lazy =

```

```

new Lazy<HeavyResourceManager>(() => new HeavyResourceManager());

public static HeavyResourceManager Instance => _lazy.Value;

private HeavyResourceManager()
{
    // Expensive initialisation: loading models, warming caches, etc.
    Console.WriteLine("HeavyResourceManager initialized.");
}

public string Process(string input) => input.ToUpper();
}

```

`Lazy<T>` is thread-safe by default. The instance is not created until the first call to `.Value` .

Side-by-Side Comparison

Feature	Static Class	Singleton (DI)
Is it an object?	No — compile-time concept	Yes — a real instance
Can implement interface?	No	Yes
Can be injected via DI?	No	Yes
Can be mocked in tests?	No	Yes
Can hold state?	Yes (shared globally, dangerous)	Yes (managed, disposable)
Inheritance?	Cannot inherit or be inherited	Can inherit and be inherited
Lifecycle control?	None — lives forever	Controlled by DI container
Thread safety?	You manage it	DI container creates once; you manage state

When to use?	Stateless utilities only	Shared stateful services
--------------	--------------------------	--------------------------

The Real-World Decision

Is the class purely stateless utility functions (like Math operations, string helpers)?

YES -> Use a static class or static methods

Does it hold state, need to be tested, need to implement an interface, or need to be swapped out?

YES -> Use a class registered as `AddSingleton()` in the DI container

In modern [ASP.NET](#) Core applications, static classes should be rare and limited to pure utilities. Anything with state or any dependency on external resources should go through the DI container as a singleton registration.

3. Lifecycle of [ASP.NET](#) Core Application

The Big Picture

When you run a .NET application, a lot happens before your first controller handles a single request. Understanding the startup lifecycle means you know exactly where to put configuration, where to register services, where to add middleware, and what runs when.

There are two phases:

1. **Build Phase** — Everything is configured and wired up
2. **Runtime Phase** — The app is running and handling requests

Phase 1: The Build Phase

Step 1: Host Creation

Everything starts with `WebApplication.CreateBuilder(args)`. This creates a `WebApplicationBuilder` which sets up:

- The underlying host (Kestrel web server)
- Default logging providers (Console, Debug, EventSource)
- Configuration sources in order:
 1. `appsettings.json`

2. `appsettings.{Environment}.json`
3. Environment variables
4. Command-line arguments
5. User Secrets (in Development)

```
var builder = WebApplication.CreateBuilder(args);  
// At this point: configuration is loaded, IConfiguration is available  
// builder.Configuration["ConnectionStrings:Default"] works here
```

Step 2: Service Registration

This is where you register everything the DI container needs to know about. The container is being built but nothing is created yet. You are just telling it: “when someone asks for `IOrderRepository` , give them `SqlOrderRepository` .”

```
// Register your own services  
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();  
builder.Services.AddSingleton<ICacheService, RedisCacheService>();  
  
// Register framework services  
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
  
// Register database context  
builder.Services.AddDbContext<AppDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));  
  
// Register authentication  
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)  
    .AddJwtBearer(options => { /* ... */ });
```

At this point, no services have been instantiated. The container just has instructions.

Step 3: Build

```
var app = builder.Build();
```

This finalises the DI container. After this point, you cannot add new service registrations. The `WebApplication` object is created. This is where the host is built and the service

provider is compiled.

Step 4: Middleware Pipeline Configuration

Now you build the HTTP pipeline — the ordered chain of middleware that every request will pass through.

```
// Order matters critically here
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
    app.UseDeveloperExceptionPage();
}

app.UseHttpsRedirection(); // Redirect HTTP to HTTPS
app.UseStaticFiles();      // Serve wwwroot files
app.UseRouting();          // Match URL to route
app.UseAuthentication();   // Who are you?
app.UseAuthorization();    // Are you allowed?
app.UseMiddleware<RequestLoggingMiddleware>(); // Custom middleware

app.MapControllers();      // Route to controller endpoints
```

The order of middleware matters. If you put `UseAuthorization()` before `UseAuthentication()`, the auth check runs without the identity being established — nothing works.

Step 5: Run

```
app.Run();
```

This starts the web server (Kestrel), starts listening on the configured ports, and blocks the main thread. The application is now live.

Phase 2: The Request Lifecycle

Once the application is running, every HTTP request goes through a defined lifecycle.

The Middleware Pipeline — In and Out

Think of middleware as a line of security checkpoints at an airport. Your request enters from the left, passes through each checkpoint, reaches the endpoint, gets a response, and then passes back through each checkpoint in reverse order.

```
Request --> [Middleware 1] --> [Middleware 2] --> [Middleware 3] --> Endpoint  
Response <-- [Middleware 1] <-- [Middleware 2] <-- [Middleware 3] <-- Endpoint
```

Each middleware can:

- Do work before calling the next middleware (inspecting headers, logging the incoming request)
- Call the next middleware (or short-circuit and return a response immediately)
- Do work after the next middleware returns (logging the response, modifying response headers)

Routing

After routing middleware matches the URL pattern, the request is directed to the appropriate controller and action method.

Model Binding

[ASP.NET](#) Core automatically extracts values from the request (route parameters, query string, body, headers, form data) and maps them to method parameters.

```
[HttpPost("orders/{id}/items")]  
public async Task<IActionResult> AddItem(  
    [FromRoute] Guid id,      // from URL path  
    [FromBody] AddItemRequest request, // from JSON body  
    [FromQuery] bool validate, // from ?validate=true  
    [FromHeader(Name = "X-Correlation-Id")] string correlationId) // from header  
{  
    // All parameters are automatically bound  
}
```

Action Execution and Filters

Before and after the action method runs, **Action Filters** execute. These are hooks for cross-cutting concerns.

Request -> Authorization Filters

- > Resource Filters (before)
- > Model Binding
- > Action Filters (before)
 - > Action Method Executes
- > Action Filters (after)
- > Result Filters (before)
 - > Result Executed (JSON serialised, response written)
- > Result Filters (after)
- > Resource Filters (after)

Filter Types

```
// Authorization Filter — runs first, can short-circuit
public class RequireAdminFilter : IAuthorizationFilter
{
    public void OnAuthorization(AuthorizationFilterContext context)
    {
        if (!context.HttpContext.User.IsInRole("Admin"))
            context.Result = new ForbidResult();
    }
}

// Action Filter — wraps the action method
public class LoggingActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        Console.WriteLine($"Action started: {context.ActionDescriptor.DisplayName}");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine($"Action finished. Exception: {context.Exception?.Message}");
    }
}

// Exception Filter — handles unhandled exceptions from actions
public class GlobalExceptionHandler : IExceptionHandler
```

```
{  
    public void OnException(ExceptionContext context)  
    {  
        context.Result = new ObjectResult(new { error = context.Exception.Message })  
        {  
            StatusCode = 500  
        };  
        context.ExceptionHandled = true;  
    }  
}
```

DI Scope During a Request

When a request arrives, **ASP.NET** Core creates a new **DI scope**. All **AddScoped** services are created fresh for this request. When the request ends, the scope is disposed — and all scoped services are disposed with it.

Request arrives

- > New DI Scope created
 - > DbContext created (Scoped)
 - > Repository created (Scoped) — gets the same DbContext
 - > Service created (Scoped) — gets the same DbContext
 - > Controller created (Scoped) — gets the Service
 - > Action executes
 - > Response written
- > DI Scope disposed
 - > DbContext.Dispose() called — connection returned to pool
 - > Repository and Service disposed

Request ends

This is why **DbContext** should always be **Scoped**. Each request gets its own context and its own database connection. No request bleeds into another.

Application Lifetime Events

You can hook into the application's start and stop events:

```
var lifetime = app.Services.GetRequiredService<IHostApplicationLifetime>();

lifetime.ApplicationStarted.Register(() =>
{
    Console.WriteLine("Application has started. Warm up caches here.");
});

lifetime.ApplicationStopping.Register(() =>
{
    Console.WriteLine("Application is stopping. Flush buffers, finish in-flight requests.");
});

lifetime.ApplicationStopped.Register(() =>
{
    Console.WriteLine("Application has stopped. Clean up resources.");
});
```

4. .NET Core vs .NET Framework

The Short Version

.NET Framework is the original, Windows-only runtime that has existed since 2002. It ships with Windows and is deeply integrated with it. .NET Core (now just called .NET from version 5 onwards) is the modern, cross-platform, open-source rebuild of .NET. It can run on Windows, Linux, and macOS.

If you are starting a new project today, you use .NET (Core). .NET Framework is for maintaining legacy systems.

The Long Version

.NET Framework — The Original

Released in 2002, .NET Framework was built to run on Windows and it was deeply integrated with Windows features — the registry, COM interop, Windows services, WCF, WebForms, WPF, WinForms.

.NET Framework

- Windows only
- Ships WITH Windows (many versions pre-installed)
- Lives at C:\Windows\Microsoft.NET\
- Tightly coupled to the OS
- Mature, stable, feature-complete
- No longer actively developed (last version: 4.8.1)
- GAC (Global Assembly Cache) for shared libraries

.NET Core / .NET — The Modern Runtime

.NET Core was released in 2016. It was a complete rewrite from scratch, designed to be:

- **Cross-platform:** Windows, Linux, macOS
- **Open source:** All code on GitHub
- **High performance:** Rebuilt from the ground up with performance in mind
- **Modular:** You reference only what you need via NuGet
- **Self-contained deployment:** You can ship the runtime with your app

.NET (Core) versions timeline:

- .NET Core 1.0 (2016) — First release
- .NET Core 2.0 (2017) — Much more complete
- .NET Core 3.0 (2019) — WinForms and WPF added (Windows only)
- .NET 5 (2020) — Merged Core and Framework branding
- .NET 6 (2021) — LTS, Minimal APIs, Hot reload
- .NET 7 (2022) — Performance improvements
- .NET 8 (2023) — LTS, Native AOT, Blazor improvements
- .NET 9 (2024) — Current

Feature-by-Feature Comparison

Feature	.NET Framework	.NET (Core)
OS Support	Windows only	Windows, Linux, macOS
Open Source	No	Yes
Performance	Good	Significantly faster
Deployment	System-wide install required	Self-contained, side-by-side

Docker support	Limited	First-class
Cloud / Kubernetes	Difficult	Native
CLI tooling	Limited (Visual Studio)	Full dotnet CLI
Startup time	Slower	Much faster
Memory footprint	Larger	Smaller
NuGet packages	Yes	Yes (modular by design)
SignalR	Yes	Yes (improved)
WCF (full server)	Yes	No (use gRPC instead)
WebForms	Yes	No
WPF/WinForms	Yes	Yes (Windows only)
gRPC	No	Yes
Minimal APIs	No	Yes (.NET 6+)
Active development	No (maintenance only)	Yes

Deployment Models

.NET Framework requires the correct version installed on the machine. Deploy your app, and if the server does not have .NET Framework 4.7.2, it fails.

.NET Core offers three models:

Framework-dependent deployment (FDD): Your app is small, but the .NET runtime must be installed on the machine.

```
dotnet publish -c Release
# Produces: MyApp.dll (small, requires runtime on machine)
```

Self-contained deployment (SCD): The runtime is bundled with your app. No installation needed on the target machine.

```
dotnet publish -c Release -r linux-x64 --self-contained
# Produces: large folder with everything needed
```

Single-file executable: Everything packaged into one file.

```
dotnet publish -c Release -r win-x64 --self-contained -p:PublishSingleFile=true  
# Produces: MyApp.exe (single executable, runs anywhere)
```

Performance Reality Check

.NET has become one of the fastest server-side platforms available. In the TechEmpower benchmarks, [ASP.NET Core](#) consistently ranks among the top frameworks globally, ahead of many Java and Node.js alternatives, and competitive with raw C/C++ in some categories.

Key performance improvements in .NET Core:

- Completely rewritten Kestrel web server (no IIS dependency)
 - `Span<T>` and `Memory<T>` for zero-allocation buffer handling
 - `System.Text.Json` — much faster than `Newtonsoft.Json`
 - Tiered JIT compilation
 - On-stack replacement and dynamic PGO (Profile-guided optimisation)
-

When Would You Still Use .NET Framework?

- You are maintaining an existing application that uses WebForms, WCF, or Windows-specific COM components
- The migration cost outweighs the benefits
- Enterprise policy requires it

For all new development, .NET 8 or .NET 9 is the answer.

5. Microservices and Communication

What Are Microservices?

A monolith is one big application. Everything — users, orders, inventory, payments, notifications — lives in one codebase, one deployable unit, one database. It is simple to start with, but as it grows it becomes slow to build, slow to deploy, and fragile.

Microservices is an architectural style where you break that monolith into a collection of small, independently deployable services. Each service:

- Owns a specific business capability
- Has its own database
- Is deployed independently
- Communicates with other services over the network

Monolith:

```
+-----+
| UserModule + OrderModule + PaymentModule |
| + InventoryModule + NotificationModule   |
|           One Database                     |
+-----+
```

Microservices:

```
+-----+ +-----+ +-----+
| User   | | Order  | | Payment |
| Service| | Service| | Service |
| DB     | | DB     | | DB      |
+-----+ +-----+ +-----+
|         | |         | |         |
+-----+ +-----+
| Inventory| | Notification|
| Service  | | Service    |
| DB       | |            |
+-----+ +-----+
```

Communication Patterns

Services need to talk to each other. There are two fundamental approaches:

synchronous and **asynchronous**.

Synchronous Communication — “I ask, I wait for the answer”

The calling service sends a request and waits for a response before it can continue.

REST over HTTP — The most common. Service A calls Service B’s API endpoint and waits.

```
// OrderService calling InventoryService via HTTP
public class InventoryClient
{
    private readonly HttpClient _httpClient;
```

```

public InventoryClient(HttpClient httpClient)
{
    _httpClient = httpClient;
}

public async Task<bool> CheckStockAsync(string productId, int quantity)
{
    var response = await _httpClient.GetAsync(
        $"http://inventory-service/api/stock/{productId}?quantity={quantity}");

    response.EnsureSuccessStatusCode();

    var result = await response.Content.ReadFromJsonAsync<StockCheckResult>();
    return result?.IsAvailable ?? false;
}
}

```

Register with the DI container:

```

builder.Services.AddHttpClient<InventoryClient>(client =>
{
    client.BaseAddress = new Uri("http://inventory-service");
    client.Timeout = TimeSpan.FromSeconds(10);
});

```

gRPC — A faster, binary protocol built on HTTP/2. Strongly typed, uses Protocol Buffers. Excellent for service-to-service calls where performance matters.

```

// inventory.proto
syntax = "proto3";
service InventoryService {
    rpc CheckStock (StockRequest) returns (StockResponse);
}
message StockRequest {
    string product_id = 1;
    int32 quantity = 2;
}
message StockResponse {
    bool is_available = 1;
}

```

```
int32 current_stock = 2;
}
```

```
// Client in OrderService
var channel = GrpcChannel.ForAddress("http://inventory-service:5001");
var client = new InventoryService.InventoryServiceClient(channel);
var response = await client.CheckStockAsync(new StockRequest
{
    ProductId = "PROD-123",
    Quantity = 5
});
```

gRPC vs REST:

Feature	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Human readable	Yes	No (binary)
Browser support	Full	Limited (needs proxy)
Performance	Good	Faster (binary, multiplexing)
Strongly typed	No (OpenAPI helps)	Yes
Streaming	Limited	Full bidirectional
Best for	Public APIs, browser clients	Internal service-to-service

Asynchronous Communication — “I send a message and move on”

Service A publishes an event. Service B (and C, and D) subscribe to it and process it in their own time. Service A does not wait. It does not even know who is listening.

This is done through a **message broker**: RabbitMQ, Azure Service Bus, Apache Kafka.

```
OrderService ---> [Message Broker] ---> InventoryService (reserves stock)
                                     ---> NotificationService (sends email)
                                     ---> AnalyticsService (records the event)
```

RabbitMQ with MassTransit in .NET:

```

// OrderService — publishing an event
public class OrderPlacedEvent
{
    public Guid OrderId { get; set; }
    public string CustomerId { get; set; }
    public decimal TotalAmount { get; set; }
}

// In PlaceOrderCommandHandler
await _publishEndpoint.Publish(new OrderPlacedEvent
{
    OrderId = order.Id,
    CustomerId = order.CustomerId,
    TotalAmount = order.TotalAmount
});
// Returns immediately. Does not wait for anyone to process it.

```

```

// NotificationService — consuming the event
public class OrderPlacedConsumer : IConsumer<OrderPlacedEvent>
{
    private readonly IEmailService _emailService;

    public OrderPlacedConsumer(IEmailService emailService)
    {
        _emailService = emailService;
    }

    public async Task Consume(ConsumeContext<OrderPlacedEvent> context)
    {
        var order = context.Message;
        await _emailService.SendOrderConfirmationAsync(order.CustomerId, order.OrderId);
    }
}

// Registration in NotificationService Program.cs
builder.Services.AddMassTransit(x =>
{
    x.AddConsumer<OrderPlacedConsumer>();
    x.UsingRabbitMq((context, cfg) =>

```

```
{
    cfg.Host("rabbitmq://localhost");
    cfg.ConfigureEndpoints(context);
});
});
```

Azure Service Bus — same concept, managed cloud service:

```
builder.Services.AddAzureServiceBus(config =>
{
    config.ConnectionString = builder.Configuration["ServiceBus:ConnectionString"];
});
```

Synchronous vs Asynchronous — When to Use Which

Scenario	Use
Need the result immediately to continue (stock check before placing order)	Synchronous (REST/gRPC)
Operation can happen in the background (sending confirmation email)	Async (Message broker)
Multiple services need to react to the same event	Async (fan-out naturally)
Strong consistency required	Synchronous
High throughput, decoupled services, resilience	Async
Simple reads (get a product by ID)	Synchronous REST
Complex workflows spanning multiple services	Async with Saga pattern

Service Discovery

In a microservices environment, services do not have fixed IP addresses. Containers spin up, scale out, and move around. Services need to discover each other dynamically.

Consul or **Kubernetes built-in service discovery** are common solutions.

In Kubernetes, every service gets a stable DNS name:

```
http://inventory-service.default.svc.cluster.local
```

With **Polly** for resilience (retries, circuit breakers):

```
builder.Services.AddHttpClient<InventoryClient>()
    .AddPolicyHandler(Policy<HttpResponseMessage>
        .Handle<HttpRequestException>()
        .OrResult(r => !r.IsSuccessStatusCode)
        .WaitAndRetryAsync(3, attempt => TimeSpan.FromSeconds(Math.Pow(2, attempt))))
    .AddPolicyHandler(Policy<HttpResponseMessage>
        .Handle<HttpRequestException>()
        .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30)));
```

This retries 3 times with exponential backoff, and if it fails 5 times in a row, opens the circuit breaker for 30 seconds (stops calling the failing service to let it recover).

API Gateway

When you have 10 microservices, the client should not call each one individually. An **API Gateway** sits in front and acts as a single entry point:

```
Client
|
v
[API Gateway]
| | | |
v v v v
[Users] [Orders] [Inventory] [Payments]
```

The gateway handles:

- Routing requests to the right service
- Authentication and authorisation (once, not in every service)
- Rate limiting
- Request aggregation (combine multiple service calls into one response)
- SSL termination

In .NET: **YARP (Yet Another Reverse Proxy)** or **Ocelot** are popular choices.

```
// YARP configuration in appsettings.json
{
  "ReverseProxy": {
    "Routes": {
      "orders-route": {
        "ClusterId": "orders-cluster",
        "Match": { "Path": "/api/orders/{**catch-all}" }
      }
    },
    "Clusters": {
      "orders-cluster": {
        "Destinations": {
          "destination1": { "Address": "http://order-service:5001" }
        }
      }
    }
  }
}
```

Saga Pattern — Distributed Transactions

You cannot use a database transaction across multiple services. If an order placement needs to: create an order, reserve inventory, and charge payment — and payment fails — how do you roll back the order and inventory?

The **Saga Pattern** solves this by choreographing a series of local transactions, each publishing events that trigger the next step. If a step fails, compensating transactions undo previous steps.

PlaceOrder Saga:

Step 1: OrderService creates order (Pending) --> publishes OrderCreated

Step 2: InventoryService reserves stock --> publishes StockReserved

Step 3: PaymentService charges card --> publishes PaymentProcessed

Step 4: OrderService marks order (Confirmed) --> done

If Step 3 fails (payment declined):

PaymentService publishes PaymentFailed

InventoryService receives it, releases reserved stock
OrderService receives it, marks order (Cancelled)

MassTransit has built-in Saga support with state machines.

6. Middleware in ASP.NET Core

What Is Middleware?

Middleware is software assembled into an application pipeline to handle requests and responses. Each piece of middleware in the pipeline:

1. Receives the HTTP context
2. Does some work
3. Either passes control to the next middleware by calling `next()`
4. Or short-circuits and writes a response directly

The pipeline is a **chain of responsibility**. Each middleware is a link.

HTTP Request

|
v

+-----+

| Exception | catches any unhandled exception from anything below
| Handler |

+-----+

|
v

+-----+

| HTTPS | redirects HTTP to HTTPS
| Redirection |

+-----+

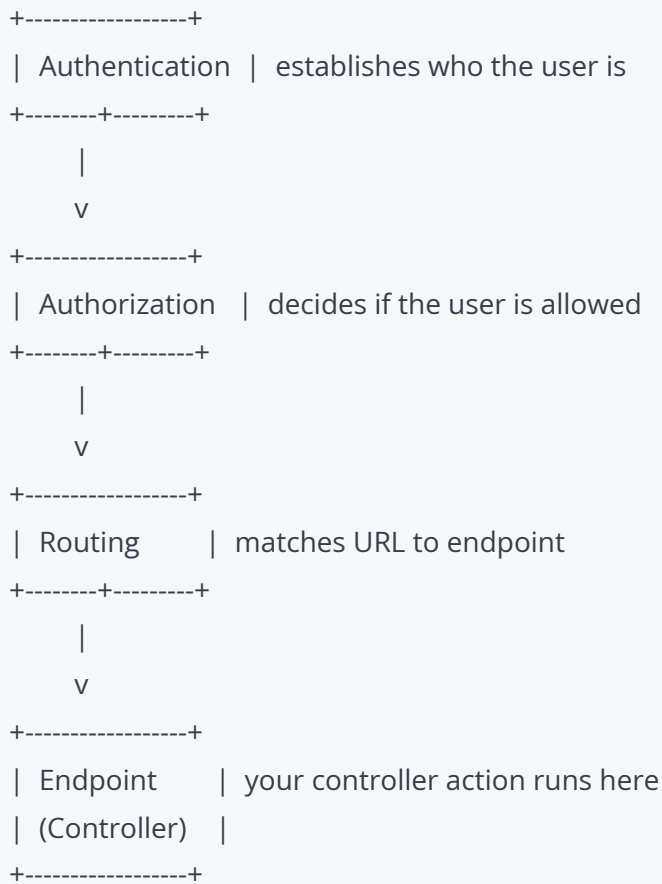
|
v

+-----+

| Static Files | serves wwwroot files, short-circuits for matches

+-----+

|
v



Built-in Middleware

```
app.UseExceptionHandler("/error"); // Global exception handling
app.UseHsts();                      // HTTP Strict Transport Security header
app.UseHttpsRedirection();          // Force HTTPS
app.UseStaticFiles();               // Serve static files from wwwroot
app.UseCors("MyPolicy");             // Cross-Origin Resource Sharing
app.UseRouting();                   // Enable endpoint routing
app.UseAuthentication();            // JWT/Cookie authentication
app.UseAuthorization();             // Policy/Role authorization
app.UseResponseCompression();       // Gzip/Brotli compression
app.UseResponseCaching();           // HTTP response caching
app.MapControllers();               // Register controller endpoints
```

Writing Custom Middleware

Convention-based middleware (the standard way):

```

public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestTimingMiddleware> _logger;

    public RequestTimingMiddleware(RequestDelegate next, ILogger<RequestTimingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var stopwatch = Stopwatch.StartNew();

        // Work BEFORE the next middleware (incoming request)
        _logger.LogInformation("Request started: {Method} {Path}",
            context.Request.Method,
            context.Request.Path);

        await _next(context); // Call the next middleware

        // Work AFTER the next middleware (outgoing response)
        stopwatch.Stop();
        _logger.LogInformation("Request finished: {StatusCode} in {Elapsed}ms",
            context.Response.StatusCode,
            stopwatch.ElapsedMilliseconds);
    }
}

// Registration
app.UseMiddleware<RequestTimingMiddleware>();

```

Interface-based middleware (cleaner, testable):

```

public class CorrelationIdMiddleware : IMiddleware
{
    private readonly ILogger<CorrelationIdMiddleware> _logger;

```

```

public CorrelationIdMiddleware ILogger<CorrelationIdMiddleware> logger)
{
    _logger = logger;
}

public async Task InvokeAsync(HttpContext context, RequestDelegate next)
{
    var correlationId = context.Request.Headers["X-Correlation-Id"].FirstOrDefault()
        ?? Guid.NewGuid().ToString();

    context.Items["CorrelationId"] = correlationId;
    context.Response.Headers["X-Correlation-Id"] = correlationId;

    using (_logger.BeginScope(new { CorrelationId = correlationId }))
    {
        await next(context);
    }
}

// Must register as a service (IMiddleware requires this)
builder.Services.AddScoped<CorrelationIdMiddleware>();
app.UseMiddleware<CorrelationIdMiddleware>();

```

Inline middleware with **Use** (for simple cases):

```

app.Use(async (context, next) =>
{
    context.Response.Headers.Add("X-Frame-Options", "DENY");
    context.Response.Headers.Add("X-Content-Type-Options", "nosniff");
    await next(context);
});

```

Short-Circuiting the Pipeline

Middleware can stop the pipeline entirely by not calling **next**. This is called short-circuiting.

```

app.Use(async (context, next) =>
{
    if (context.Request.Headers["X-API-Key"] != "secret-key")
    {
        context.Response.StatusCode = 401;
        await context.Response.WriteAsync("Unauthorized: invalid API key.");
        return; // Does NOT call next — pipeline stops here
    }

    await next(context); // Valid request — continue
});

```

`app.Run()` always short-circuits — it is a terminal middleware:

```

app.Run(async context =>
{
    await context.Response.WriteAsync("This is the end of the pipeline.");
    // No next() call — this is always the last stop
});

```

Real-World Middleware Examples

Global Exception Handling Middleware:

```

public class GlobalExceptionHandler : IMiddleware
{
    private readonly ILogger<GlobalExceptionHandler> _logger;

    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger)
    {
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context, RequestDelegate next)
    {
        try
        {
            await next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "An unhandled exception occurred.");
        }
    }
}

```

```

    }
    catch (ValidationException ex)
    {
        _logger.LogWarning(ex, "Validation error for request {Path}", context.Request.Path);
        context.Response.StatusCode = 400;
        context.Response.ContentType = "application/json";
        await context.Response.WriteAsJsonAsync(new
        {
            error = "Validation failed",
            details = ex.Errors
        });
    }
    catch (NotFoundException ex)
    {
        context.Response.StatusCode = 404;
        await context.Response.WriteAsJsonAsync(new { error = ex.Message });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Unhandled exception for request {Path}", context.Request.Path);
        context.Response.StatusCode = 500;
        await context.Response.WriteAsJsonAsync(new
        {
            error = "An unexpected error occurred."
        });
    }
}
}
}

```

Request/Response Logging Middleware:

```

public class RequestResponseLoggingMiddleware : IMiddleware
{
    private readonly ILogger<RequestResponseLoggingMiddleware> _logger;

    public RequestResponseLoggingMiddleware(ILogger<RequestResponseLoggingMiddleware> logger)
    {
        _logger = logger;
    }
}

```

```

public async Task InvokeAsync(HttpContext context, RequestDelegate next)
{
    // Buffer the request body so we can read it
    context.Request.EnableBuffering();
    var requestBody = await new StreamReader(context.Request.Body).ReadToEndAsync();
    context.Request.Body.Position = 0; // Reset for the next middleware

    _logger.LogInformation("HTTP {Method} {Path} | Body: {Body}",
        context.Request.Method,
        context.Request.Path,
        requestBody.Length > 500 ? requestBody[..500] + "..." : requestBody);

    // Capture response body
    var originalBody = context.Response.Body;
    using var responseBuffer = new MemoryStream();
    context.Response.Body = responseBuffer;

    await next(context);

    responseBuffer.Position = 0;
    var responseBody = await new StreamReader(responseBuffer).ReadToEndAsync();

    _logger.LogInformation("HTTP {StatusCode} | Body: {Body}",
        context.Response.StatusCode,
        responseBody.Length > 500 ? responseBody[..500] + "..." : responseBody);

    responseBuffer.Position = 0;
    await responseBuffer.CopyToAsync(originalBody);
    context.Response.Body = originalBody;
}
}

```

Middleware Ordering — Why It Matters

The order is not arbitrary. It follows the principle that earlier middleware can protect or prepare for later middleware.

```

// CORRECT ORDER
app.UseExceptionHandler(); // Must be first — catches exceptions from everything below

```

```

app.UseHttpsRedirection(); // Redirect before doing any real work
app.UseStaticFiles();      // Serve static files before authentication (they are public)
app.UseRouting();          // Routing must be before authentication for route-based auth
app.UseAuthentication();   // Establish identity before checking permissions
app.UseAuthorization();    // Check permissions after identity is established
app.MapControllers();      // Endpoints last

```

What breaks if you get the order wrong:

- `UseAuthorization()` before `UseAuthentication()` : the user has no identity when authorisation runs, so everyone gets 401
- `UseStaticFiles()` after `UseAuthentication()` : static files require authentication, which is usually wrong
- `UseExceptionHandler()` in the middle: exceptions from middleware above it are not caught

Map , MapWhen , and UseWhen — Branching the Pipeline

Sometimes you want different middleware for different paths.

```

// Completely separate pipeline branch for /api/* routes
app.Map("/api", apiApp =>
{
    apiApp.UseAuthentication();
    apiApp.UseAuthorization();
    apiApp.MapControllers();
});

// Conditional middleware based on a predicate
app.UseWhen(context => context.Request.Path.StartsWithSegments("/admin"), adminApp =>
{
    adminApp.UseMiddleware<AdminAuditMiddleware>();
});

// MapWhen creates a completely separate branch (does not rejoin main pipeline)
app.MapWhen(context => context.Request.Headers.ContainsKey("X-Internal"), internalApp =>
{
    internalApp.UseMiddleware<InternalRequestMiddleware>();
});

```

```
internalApp.MapControllers();  
});
```

7. Interview Questions and Answers

Clean Architecture

Q: What is the Dependency Rule in Clean Architecture?

The Dependency Rule states that source code dependencies must point only inward — toward higher-level, more abstract policies. The Domain layer has no dependencies on any other layer. The Application layer depends only on the Domain. Infrastructure depends on Domain and Application. The Presentation layer depends on Application. Nothing in the inner layers knows about the outer layers. This ensures that changing a database or swapping a framework does not affect business logic.

Q: What is the difference between Clean Architecture and a traditional N-Tier architecture?

In traditional N-Tier (Presentation -> Business Logic -> Data Access -> Database), the business logic depends on the data access layer — meaning the database is at the center. In Clean Architecture, business logic is at the center and nothing depends on the database. The database depends on the domain through interfaces, which is the inverse. Clean Architecture is inside-out compared to N-Tier.

Q: What is CQRS and how does it fit with Clean Architecture?

CQRS (Command Query Responsibility Segregation) separates operations that change state (Commands) from operations that read state (Queries). It fits naturally in the Application layer of Clean Architecture — Commands go through the full domain logic, while Queries can bypass the domain and read directly from the database in an optimised way. MediatR is commonly used as the in-process dispatcher.

Q: How would you test business logic in Clean Architecture without a database?

Because the Domain and Application layers have no infrastructure dependencies, you can test them with just a unit test project. You create in-memory fake implementations of the interfaces (like `IOrderRepository`) and inject them. No database, no HTTP, no configuration required. This is one of the biggest selling points of Clean Architecture.

Singleton vs Static

Q: Can a singleton be mocked in unit tests? Can a static class?

A singleton registered through the DI container implements an interface. You can inject a mock implementation in tests by registering the mock instead of the real class. A static class cannot implement an interface and cannot be injected, so it cannot be mocked. This is why static classes make code hard to test.

Q: What is thread safety in a singleton and how do you handle it?

Since a singleton is shared across all threads, any mutable state must be protected. Options include using `lock` for exclusive access, using `ConcurrentDictionary` or other thread-safe collections, using `Interlocked` for atomic operations on primitives, or designing the singleton to be stateless or immutable after construction. The safest approach is to make singletons stateless or only initialise state in the constructor.

Q: What is Lazy initialisation and when would you use it in a singleton?

`Lazy<T>` defers the creation of an object until it is first accessed. Use it in a singleton when the initialisation is expensive and the object might not be needed for every application run. `Lazy<T>` is thread-safe by default using double-checked locking internally.

ASP.NET Core Lifecycle

Q: What is the order of middleware execution? Is it the same for requests and responses?

For requests, middleware executes in the order it is registered — top to bottom. For responses, middleware executes in reverse order — bottom to top. This is because each middleware calls `next()`, which goes deeper into the pipeline. On the way back, control returns to each middleware in reverse. The first middleware registered has the last word on the outgoing response.

Q: What is a DI scope in the context of an HTTP request?

ASP.NET Core creates a new DI scope for every HTTP request. All services registered as `AddScoped` are created once per scope (per request) and disposed when the scope ends (when the response is sent). This is why `DbContext` is scoped — each request gets its own database context and connection.

Q: What is the difference between `app.Use()` , `app.Run()` , and `app.Map()` ?

`app.Use()` adds middleware that calls `next()` to pass control to the next middleware in the pipeline. `app.Run()` adds terminal middleware that does not call `next()` — it always ends the pipeline. `app.Map()` creates a branch of the pipeline for a specific URL path prefix — requests matching that path go through the branch and not the main pipeline.

Q: What are Action Filters and how are they different from middleware?

Both intercept request processing, but at different levels. Middleware operates on raw HTTP context — it is part of the request pipeline and runs for every request including static files, swagger, etc. Action Filters are part of the MVC framework and run only for controller actions. They have access to MVC-specific concepts like `ActionContext` , model binding results, and the controller instance. Middleware is for infrastructure concerns (logging, auth, exceptions). Filters are for MVC-specific concerns (validation, response transformation for specific actions).

.NET Core vs .NET Framework

Q: Can you run .NET Framework code on Linux?

No. .NET Framework is Windows-only. To run C# code on Linux, you must use .NET Core (.NET 5 and later). Some .NET Framework code can be migrated with minimal changes; other parts that rely on Windows-specific APIs (COM, the Windows Registry, WCF server) require rework.

Q: What is self-contained deployment in .NET Core?

A self-contained deployment bundles the .NET runtime along with your application code into a single publishable package. The target machine does not need .NET installed. This is useful for server environments where you control what is deployed, and for avoiding version conflicts between applications. The trade-off is a larger deployment size.

Q: Why is **ASP.NET Core faster than classic **ASP.NET**?**

ASP.NET Core uses the Kestrel server built from scratch for performance, replacing the old IIS-dependent pipeline. It uses `Span<T>` for zero-copy buffer operations, has a more efficient routing system, uses `System.Text.Json` which is significantly faster than Newtonsoft, and the middleware pipeline has much less overhead than the old HTTP Modules and HTTP Handlers model.

Microservices

Q: When would you choose gRPC over REST for microservice communication?

Choose gRPC for internal service-to-service communication where performance matters — it is faster due to binary serialisation and HTTP/2 multiplexing, and it is strongly typed via Protocol Buffers which eliminates a class of serialisation bugs. Use REST for public-facing APIs, browser clients, or when human-readability and broad ecosystem compatibility are more important than raw performance.

Q: What is the difference between synchronous and asynchronous communication in microservices?

Synchronous communication (REST, gRPC) means the caller waits for a response. This is simple but creates temporal coupling — if the downstream service is slow or down, the caller is affected. Asynchronous communication uses a message broker. The sender publishes a message and moves on. The receiver processes it independently. This decouples services in time, improves resilience, and allows natural fan-out (multiple services react to one event), but adds complexity.

Q: What is a captive dependency in microservices communication?

In the DI context, it is a service with a longer lifetime holding a reference to one with a shorter lifetime. In the microservices context, a more common concern is a service being so tightly coupled to another (via synchronous calls through multiple layers) that it creates a chain of failures. This is addressed with circuit breakers (Polly), bulkhead patterns, and preferring async communication for operations that do not need immediate results.

Q: What is the Saga pattern and why is it needed in microservices?

In a distributed system, you cannot use a single database transaction across multiple services. The Saga pattern manages distributed transactions as a sequence of local transactions, each triggering the next via events. If any step fails, compensating transactions roll back the previous steps. There are two styles: Choreography (each service reacts to events from the previous service) and Orchestration (a central saga orchestrator tells each service what to do).

Middleware

Q: What is the difference between `IMiddleware` and convention-based middleware?

Convention-based middleware resolves its dependencies from the constructor, which means they are resolved once when the middleware is instantiated (singleton-like).

`IMiddleware` is registered as a service and its `InvokeAsync` method receives dependencies as parameters, so scoped dependencies can be injected per request. For middleware that needs scoped services (like a `DbContext`), use `IMiddleware` .

Q: How would you add a global exception handler using middleware?

Create a class implementing `IMiddleware` with a try-catch wrapping the `await next(context)` call. In the catch blocks, set the appropriate HTTP status codes and write a JSON error response. Register the middleware as a service and add it as the first middleware in the pipeline so it catches exceptions from all middleware below it.

Q: Can middleware access the DI container?

Yes. Convention-based middleware can receive singleton services via constructor injection. For scoped services, inject them into the `InvokeAsync` method as parameters — [ASP.NET Core](#) resolves them from the request scope. Using `IMiddleware` is the cleaner approach for scoped dependencies because the middleware itself is resolved from the DI container per-request.

Quick Reference Cheat Sheet

Clean Architecture Layer Dependencies

```
Domain      <- depends on nothing
Application <- depends on Domain only
Infrastructure <- depends on Domain + Application
Presentation <- depends on Application only
```

Singleton vs Static — Decision Tree

```
Pure stateless utility with no external deps?
  YES -> static class or static methods
  NO  -> registered service
```

Does it need to implement an interface?

YES -> DI-registered singleton

Does it need to be mocked in tests?

YES -> DI-registered singleton

Does it have shared state?

YES, immutable after init -> DI-registered singleton, safe

YES, mutable -> DI-registered singleton, add thread safety

Middleware Order (Always Follow This)

1. Exception Handler (catch everything below)
2. HSTS / HTTPS Redirect
3. Static Files (no auth needed)
4. Routing
5. CORS
6. Authentication (who are you)
7. Authorization (what can you do)
8. Custom Middleware
9. Endpoints / Controllers

Microservice Communication Quick Guide

Need immediate response? -> REST or gRPC

Multiple services react to event? -> Message broker (async)

Performance-critical internal? -> gRPC

Public API / browser? -> REST

Long-running distributed workflow -> Saga pattern

.NET Runtime Quick Pick

New project? -> .NET 8 or .NET 9

Need Linux / Docker / Cloud? -> .NET 8 or .NET 9

Maintaining old app?	-> Stay on .NET Framework until migration budget exists
Windows desktop app?	-> .NET 8 (WinForms/WPF supported)

Master these six areas and you will handle any mid-to-senior level .NET interview with confidence. The patterns here are not academic — they appear in every production .NET codebase worth working on.